Labo2 Geavanceerde Computer Architecture

Arne Van der Stede en Syme Vandenbosch

Academiejaar 2025-2026
February 26, 2026

1 Introduction

Deze labo opdracht bestaat eruit om op verschillende manieren de maximale waarde van een 1D array te bepalen. Waardoor we de prestaties van een CPU kunnen vergelijken met de GPU.

Er worden drie verschillende manieren uitgevoerd:

- een sequentiële CPU implementatie met een geneste for lus
- een GPU die gebruikmaakt van atomaire operaties
- een GPU die gebaseerd is op parallelle reductie

Door het vergelijken van de uitvoeringstijd krijgen we een zicht op de schaalbaarheid en overhead van parallelisatie en wat de efficiëntste manier is om deze opdracht uit te voeren.

Thuis input sizes variëren en in kaart brengen

2 Code

2.1 CPU

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>
#include <vector>

int main() {

    std::cout << "# Numbers | Time (ms)| Max Value\n";
    for (long long N = 10; N <= 100000000; N *= 10) {
        std::vector<int> arr(N);
        for (long long i = 0; i < N; i++) {
            arr[i] = rand();
        }

        const int runs = 100;
        auto start = std::chrono::high_resolution_clock::now();

        int max_val = 0;
        for (int run = 0; run < runs; run++) {
            max_val = arr[0];
            for (long long i = 1; i < N; i++) {
                if (arr[i] > max_val) {
                    max_val = arr[i];
                }
            }
        }

        auto end = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double, std::milli> duration = end - start;
        double avg_ms = duration.count() / runs;

```

```
        std::cout << std::fixed << std::setprecision(6);
        std::cout << N << ", " << avg_ms << ", " << max_val << std::endl;
    }

    return 0;
}
```

Listing 1: max detection using cpu

2.2 Atomic operations

```
#include <iostream>
#include <cuda_runtime.h>
#include <cstdlib>
#include <climits>
#include <iomanip>
#include <vector>

__global__ void max_kernel(int *arr, int N, int *result) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < N) {
        atomicMax(result, arr[idx]);
    }
}

int main() {

    int blocks = 0;
    int threadsPerBlock = 128;

    std::cout << "# Numbers | Time (ms) | GPU Max | CPU Max\n";

    for (long long N = 10; N <= 100000000; N *= 10) {
        std::vector<int> arr_in(N);
        int cpu_max = INT_MIN;
        for (long long i = 0; i < N; i++) {
            arr_in[i] = rand();
            if (arr_in[i] > cpu_max) cpu_max = arr_in[i];
        }

        int *d_in, *d_result;
        cudaMalloc((void**)&d_in, N * sizeof(int));
        cudaMalloc((void**)&d_result, sizeof(int));

        cudaMemcpy(d_in, arr_in.data(), N * sizeof(int), cudaMemcpyHostToDevice);

        const int runs = 100;
        cudaEvent_t start_cuda, stop_cuda;
        cudaEventCreate(&start_cuda);
        cudaEventCreate(&stop_cuda);

        float total_ms = 0;
        int gpu_max = 0;
        blocks = (N + threadsPerBlock - 1) / threadsPerBlock;

        for (int run = 0; run < runs; run++) {
            int init = INT_MIN;
            cudaMemcpy(d_result, &init, sizeof(int), cudaMemcpyHostToDevice);

            cudaEventRecord(start_cuda);

            max_kernel<<<blocks, threadsPerBlock>>>(d_in, N, d_result);

            cudaEventRecord(stop_cuda);
            cudaEventSynchronize(stop_cuda);

            float ms;
            cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
            total_ms += ms;
```

```
        }

        cudaMemcpy(&gpu_max, d_result, sizeof(int), cudaMemcpyDeviceToHost);

        std::cout << std::fixed << std::setprecision(6);
        std::cout << N << ", " << (total_ms / runs) << ", " << gpu_max << ", " << cpu_max <<
            std::endl;
        std::cout << "Blocks: " << blocks << " | Threads per block: " << threadsPerBlock <<
            std::endl;

        cudaFree(d_in);
        cudaFree(d_result);
        cudaEventDestroy(start_cuda);
        cudaEventDestroy(stop_cuda);
    }

    return 0;
}
```

Listing 2: max detection using atomic operations

2.3 Reduction

```
#include <iostream>
#include <cuda_runtime.h>
#include <cstdlib>
#include <climits>
#include <iomanip>
#include <vector>

__global__ void max_reduction(const int *in, int *out, int N) {
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;

    int val = (idx < N) ? in[idx] : INT_MIN;
    sdata[tid] = val;
    __syncthreads();

    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            sdata[tid] = max(sdata[tid], sdata[tid + stride]);
        }
        __syncthreads();
    }

    if (tid == 0)
        out[blockIdx.x] = sdata[0];
}

int main() {
    int threadsPerBlock = 512;

    std::cout << "# Numbers | Time (ms) | GPU Max | CPU Max\n";

    for (long long N = 10; N <= 100000000; N *= 10) {
        std::vector<int> h_in(N);
        int cpu_max = INT_MIN;
        for (long long i = 0; i < N; i++) {
            h_in[i] = rand();
            if (h_in[i] > cpu_max) cpu_max = h_in[i];
        }

        int *d_buf1, *d_buf2;
        cudaMalloc(&d_buf1, N * sizeof(int));
        cudaMalloc(&d_buf2, ((N + threadsPerBlock - 1) / threadsPerBlock) * sizeof(int));

        cudaMemcpy(d_buf1, h_in.data(), N * sizeof(int), cudaMemcpyHostToDevice);

```

```
        const int runs = 100;
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        float total_ms = 0.0f;
        int gpu_max = 0;
        int first_pass_blocks = (N + threadsPerBlock - 1) / threadsPerBlock;

        for (int r = 0; r < runs; r++) {
            long long n = N;
            int *in = d_buf1;
            int *out = d_buf2;

            cudaEventRecord(start);

            while (n > 1) {
                int blocks = (n + threadsPerBlock - 1) / threadsPerBlock;
                size_t shmem = threadsPerBlock * sizeof(int);
                max_reduction<<<blocks, threadsPerBlock, shmem>>>(in, out, (int)n);
                n = blocks;
                std::swap(in, out);
            }

            cudaEventRecord(stop);
            cudaEventSynchronize(stop);

            float ms;
            cudaEventElapsedTime(&ms, start, stop);
            total_ms += ms;
        }

        // Final pass to get the value (not timed)
        long long n_val = N;
        int *in_val = d_buf1;
        int *out_val = d_buf2;
        while (n_val > 1) {
            int blocks = (n_val + threadsPerBlock - 1) / threadsPerBlock;
            max_reduction<<<blocks, threadsPerBlock, threadsPerBlock * sizeof(int)>>>(in_val,
                out_val, (int)n_val);
            n_val = blocks;
            std::swap(in_val, out_val);
        }
        cudaMemcpy(&gpu_max, in_val, sizeof(int), cudaMemcpyDeviceToHost);

        std::cout << std::fixed << std::setprecision(6);
        std::cout << N << ", " << total_ms / runs << ", " << gpu_max << ", " << cpu_max <<
            std::endl;
        std::cout << "Blocks: " << first_pass_blocks << " | Threads per block: " <<
            threadsPerBlock << std::endl;

        cudaFree(d_buf1);
        cudaFree(d_buf2);
        cudaEventDestroy(start);
        cudaEventDestroy(stop);
    }

    return 0;
}
```

Listing 3: max detection using reduction

3 Results

3.1 Max detection using CPU

```
# Numbers | Time (ms)| Max Value
10, 0.000004, 1957747793
100, 0.000045, 2145174067
1000, 0.000486, 2147469841
10000, 0.006817, 2146830235
100000, 0.055104, 2147466242
1000000, 0.564419, 2147480021
10000000, 6.102006, 2147483025
100000000, 61.950644, 2147483611
```

Figure 1: Enter Caption

3.2 Max detection using atomic functions

32 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.101983, 1957747793, 1957747793
Blocks: 1 | Threads per block: 32
100, 0.034525, 2145174067, 2145174067
Blocks: 4 | Threads per block: 32
1000, 0.009067, 2147469841, 2147469841
Blocks: 32 | Threads per block: 32
10000, 0.100770, 2146830235, 2146830235
Blocks: 313 | Threads per block: 32
100000, 0.076638, 2147466242, 2147466242
Blocks: 3125 | Threads per block: 32
1000000, 0.172711, 2147480021, 2147480021
Blocks: 31250 | Threads per block: 32
10000000, 1.636541, 2147483025, 2147483025
Blocks: 312500 | Threads per block: 32
100000000, 10.792768, 2147483611, 2147483611
Blocks: 3125000 | Threads per block: 32
```

Figure 2: Enter Caption

64 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.014989, 1957747793, 1957747793
Blocks: 1 | Threads per block: 64
100, 0.008864, 2145174067, 2145174067
Blocks: 2 | Threads per block: 64
1000, 0.010459, 2147469841, 2147469841
Blocks: 16 | Threads per block: 64
10000, 0.009149, 2146830235, 2146830235
Blocks: 157 | Threads per block: 64
100000, 0.012332, 2147466242, 2147466242
Blocks: 1563 | Threads per block: 64
1000000, 0.071484, 2147480021, 2147480021
Blocks: 15625 | Threads per block: 64
10000000, 0.635981, 2147483025, 2147483025
Blocks: 156250 | Threads per block: 64
100000000, 4.967771, 2147483611, 2147483611
Blocks: 1562500 | Threads per block: 64
```

Figure 3: Enter Caption

128 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.013551, 1957747793, 1957747793
Blocks: 1 | Threads per block: 128
100, 0.007940, 2145174067, 2145174067
Blocks: 1 | Threads per block: 128
1000, 0.007831, 2147469841, 2147469841
Blocks: 8 | Threads per block: 128
10000, 0.009662, 2146830235, 2146830235
Blocks: 79 | Threads per block: 128
100000, 0.013612, 2147466242, 2147466242
Blocks: 782 | Threads per block: 128
1000000, 0.065878, 2147480021, 2147480021
Blocks: 7813 | Threads per block: 128
10000000, 0.590215, 2147483025, 2147483025
Blocks: 78125 | Threads per block: 128
100000000, 4.911538, 2147483611, 2147483611
Blocks: 781250 | Threads per block: 128
```

Figure 4: Enter Caption

256 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.053550, 1957747793, 1957747793
Blocks: 1 | Threads per block: 256
100, 0.108160, 2145174067, 2145174067
Blocks: 1 | Threads per block: 256
1000, 0.108791, 2147469841, 2147469841
Blocks: 4 | Threads per block: 256
10000, 0.048860, 2146830235, 2146830235
Blocks: 40 | Threads per block: 256
100000, 0.050509, 2147466242, 2147466242
Blocks: 391 | Threads per block: 256
1000000, 0.114614, 2147480021, 2147480021
Blocks: 3907 | Threads per block: 256
10000000, 0.614059, 2147483025, 2147483025
Blocks: 39063 | Threads per block: 256
100000000, 5.072584, 2147483611, 2147483611
Blocks: 390625 | Threads per block: 256
```

Figure 5: Enter Caption

512 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.013670, 1957747793, 1957747793
Blocks: 1 | Threads per block: 512
100, 0.007208, 2145174067, 2145174067
Blocks: 1 | Threads per block: 512
1000, 0.009131, 2147469841, 2147469841
Blocks: 2 | Threads per block: 512
10000, 0.008623, 2146830235, 2146830235
Blocks: 20 | Threads per block: 512
100000, 0.012969, 2147466242, 2147466242
Blocks: 196 | Threads per block: 512
1000000, 0.144703, 2147480021, 2147480021
Blocks: 1954 | Threads per block: 512
10000000, 0.659170, 2147483025, 2147483025
Blocks: 19532 | Threads per block: 512
100000000, 5.375719, 2147483611, 2147483611
Blocks: 195313 | Threads per block: 512
```

Figure 6: Enter Caption

3.3 Max detection using reduction

32 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.020160, 1957747793, 1957747793
Blocks: 1 | Threads per block: 32
100, 0.014002, 2145174067, 2145174067
Blocks: 4 | Threads per block: 32
1000, 0.086139, 2147469841, 2147469841
Blocks: 32 | Threads per block: 32
10000, 0.029934, 2146830235, 2146830235
Blocks: 313 | Threads per block: 32
100000, 0.049652, 2147466242, 2147466242
Blocks: 3125 | Threads per block: 32
1000000, 0.272019, 2147480021, 2147480021
Blocks: 31250 | Threads per block: 32
10000000, 2.168827, 2147483025, 2147483025
Blocks: 312500 | Threads per block: 32
100000000, 15.073801, 2147483611, 2147483611
Blocks: 3125000 | Threads per block: 32
```

Figure 7: Enter Caption

64 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.016137, 1957747793, 1957747793
Blocks: 1 | Threads per block: 64
100, 0.014409, 2145174067, 2145174067
Blocks: 2 | Threads per block: 64
1000, 0.016490, 2147469841, 2147469841
Blocks: 16 | Threads per block: 64
10000, 0.021890, 2146830235, 2146830235
Blocks: 157 | Threads per block: 64
100000, 0.037074, 2147466242, 2147466242
Blocks: 1563 | Threads per block: 64
1000000, 0.151574, 2147480021, 2147480021
Blocks: 15625 | Threads per block: 64
10000000, 1.302217, 2147483025, 2147483025
Blocks: 156250 | Threads per block: 64
100000000, 8.990037, 2147483611, 2147483611
Blocks: 1562500 | Threads per block: 64
```

Figure 8: Enter Caption

128 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.015132, 1957747793, 1957747793
Blocks: 1 | Threads per block: 128
100, 0.008035, 2145174067, 2145174067
Blocks: 1 | Threads per block: 128
1000, 0.017280, 2147469841, 2147469841
Blocks: 8 | Threads per block: 128
10000, 0.016398, 2146830235, 2146830235
Blocks: 79 | Threads per block: 128
100000, 0.030247, 2147466242, 2147466242
Blocks: 782 | Threads per block: 128
1000000, 0.173296, 2147480021, 2147480021
Blocks: 7813 | Threads per block: 128
10000000, 1.268705, 2147483025, 2147483025
Blocks: 78125 | Threads per block: 128
100000000, 8.515145, 2147483611, 2147483611
Blocks: 781250 | Threads per block: 128
```

Figure 9: Enter Caption

256 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.013767, 1957747793, 1957747793
Blocks: 1 | Threads per block: 256
100, 0.008038, 2145174067, 2145174067
Blocks: 1 | Threads per block: 256
1000, 0.013111, 2147469841, 2147469841
Blocks: 4 | Threads per block: 256
10000, 0.011931, 2146830235, 2146830235
Blocks: 40 | Threads per block: 256
100000, 0.033558, 2147466242, 2147466242
Blocks: 391 | Threads per block: 256
1000000, 0.158741, 2147480021, 2147480021
Blocks: 3907 | Threads per block: 256
10000000, 1.336814, 2147483025, 2147483025
Blocks: 39063 | Threads per block: 256
100000000, 9.670497, 2147483611, 2147483611
Blocks: 390625 | Threads per block: 256
```

Figure 10: Enter Caption

512 threads

```
# Numbers | Time (ms) | GPU Max | CPU Max
10, 0.021772, 1957747793, 1957747793
Blocks: 1 | Threads per block: 512
100, 0.064824, 2145174067, 2145174067
Blocks: 1 | Threads per block: 512
1000, 0.024419, 2147469841, 2147469841
Blocks: 2 | Threads per block: 512
10000, 0.018555, 2146830235, 2146830235
Blocks: 20 | Threads per block: 512
100000, 0.100530, 2147466242, 2147466242
Blocks: 196 | Threads per block: 512
1000000, 0.184140, 2147480021, 2147480021
Blocks: 1954 | Threads per block: 512
10000000, 1.662805, 2147483025, 2147483025
Blocks: 19532 | Threads per block: 512
100000000, 10.600451, 2147483611, 2147483611
Blocks: 195313 | Threads per block: 512
```

Figure 11: Enter Caption

4 Guiding Questions

4.1 Is the result correct?

We controleren de resultaten door ze telkens te vergelijken met de waarde die door de CPU bepaald is. We krijgen hierbij voor zowel reduction als atomic de correcte resultaten zoals te zien is in de bijgevoegde outputs.

4.2 How does time increase as a function of input size?

Het toenemen van de uitvoeringstijd i.f.v. de inputgroote hangt grotendeels af van de gebruikte aanpak.

Voor de CPU versie stijgt de uitvoeringstijd min of meer linair met het aantal elementen. Er telkens per verhoging in factor tien ook een tijdsverhoging van een factor tien. Tussen de 10 000 en de 100 000 elementen is er een lichte toename per element omdat niet alle elementen nog in de cache passen.

bron: `https://www.youtube.com/watch?v=gvbQjHmHIAg` (7:14 min)

Bij de GPU versie met atomic operations stijgt de tijd ook lineair met het aantal elementen. Als er veel threads tegelijk een *atomicMax()* functie uitvoeren dan ontstaan er contentions (schrijven op dezelfde geheugenlocatie). Wat bij vooral kleine array's kan leiden schaalbaarheidsproblemen hier is de tijdswinst voor de GPU dan ook kleiner dan bij de CPU. De winst is pas zichtbaar vanaf 100 000. Bij grote array's is de snelheidswintst pas duidelijk. Bij 128 threads per block komen we de beste waarde uit.

bron: `https://supercomputingblog.com/cuda/cuda-tutorial-5-performance-of-atomics/`

Bij de GPU versie met reductie is de schaalbaarheid het beste. Het zoeken naar het maximum wordt verdeeld over veel threads en gecombineerd in reductiestappen. Dit leidt tot het feit dat de tijd lineair groeit met het aantal inputs. Maar de constante factor is veel kleiner dankzij het parallelisme. Hoewel het in theorie het een beter schaalbaarheid zou moeten geven. Leidt deze versie voor meerdere kernel launches en zorgt dus voor overhead die de prestaties dus slechter maken. Bij grote arrays is deze versie trager dan de naïve atomicMax (8.5 ms vs 4.9 ms bij 100M).

bron: `https://stackoverflow.com/questions/5923978/cuda-reduction-or-atomic-operations`

4.3 What happens if the input is larger than 2048 (how would you ensure correct calculation and are there additional synchronization steps)?

Als de input groter dan 2048 elementen wordt dan moet de berekening verdeeld worden over meerdere blokken (aangezien een blok slechts uit 1024 treads bestaat). Er bestaat geen mogelijkheid om blokken onderling met elkaar te synchroniseren binnen één kernel. Er is dus een extra stap nodig voor de synchronisatie.

Dit kan door in twee stappen reductie uit te voeren. Waarbij eerst per blok een lokaal maximum berekend wordt en het resultaat naar de een globaal array wegschrijft. Vervolgens kan in een tweede kernel opnieuw reductie uitgevoerd worden op de globale array om daar dan het algemene maximum te bepalen. De verschillende kernels zorgen voor synchronisatie.

Elk blok kan zijn lokaal maximum via een *atomicMax()* functie bepalen en naar een globale variabele schrijven. Er is wel prestatieverlies dat te wijten is aan het feit dat wanneer meerdere blokken tegelijk proberen schrijven naar de globale variabele.

bron: `https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf`

4.4 Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?

Als er minder threads gebruikt worden dan dat er data elementen zijn. Dan moet een thread meerdere elementen verwerken. Door het gebruik te maken van een grid-stride loop.

Dit houdt in dat elke thread begint bij een startindex. Vervolgens springt het verder met een vaste stap (stride). Op deze manier dekken alle threads alle elementen af. vb. (thread 0 verwerkt 0, 100, 200, ... en thread 1 verwerkt 1, 101, 102, ...).

bron: `https://developer.nvidia.com/blog/cuda-pro-tip-write-flexible-kernels-grid-stride-loops/`

4.5 How would you tackle a 2D array?

Er zijn twee manieren om dit uit te voeren. Ofwel kies je voor de 2D array te behandelen als een grote 1D array. Ofwel wordt er gebruik gemaakt van een 2D grid en 2D blokken. Er verandert dus hoofdzakelijk iets in de manier van indexeren niet echt in het algoritme.

Een 2D array wordt in het geheugen lineair opgeslagen hierdoor kan het als een 1D array behandeld worden. Elke thread berekent een globale index en verwerkt afhankelijk van het aantal thread t.o.v. het aantal element één of meerdere elementen.

De andere optie is door gebruik te maken van een 2D grid en blokstructuur. Elke thread krijgt een rij- en kolomindex toegewezen. Dit is een meer overzichtelijke manier aangezien dit beter aan sluit bij de aangeleverde 2D array. Wederom als het aantal elementen groter wordt dan het aantal threads dan kunnen de threads weer meerdere elementen verwerken via een grid-stride loop.

Om het maximum te bepalen blijft het reductieprincipe hetzelfde als bij de 1D array (thread bepaald lokaal max. daarna volgt reductie binnen het blok en dan uiteindelijk wordt het globale max. bepaald via atomaire operaties).

De 1D-benadering met grid-stride is vaak sneller en eenvoudiger te implementeren. De 2D grid/blocks is duidelijker en beter voor coalesced memory access bij row-major toegang.

bronen : `https://stackoverflow.com/questions/14100723/creation-2d-grid-in-cuda-for-gpgpu-using-c` / `https://stackoverflow.com/questions/30853992/when-do-we-need-two-dimension-threads-in-cuda`

5 Conclusie

In deze labo-opdracht hebben we drie verschillende methodes vergeleken voor het bepalen van het maximum van een array: een sequentiële CPU-implementatie, een GPU-implementatie met atomic operations en een GPU-implementatie op basis van parallelle reductie.

De metingen tonen een duidelijk verschil in de prestaties. De CPU implementatie heeft ongeveer 62 ms (gemiddeld over 100 runs). De GPU met atomic operations bereikt een gemiddelde tijd van ongeveer 4.9 ms (bij 128 threads per block). Wat een versnelling van een factor 12–13 is ten opzichte van de CPU. En de reductie versie presteert trager met ongeveer 8.5 ms (bij 128 threads per block). Wat nog steeds een factor 7–8 rapper is dan de CPU, maar ongeveer 1.7 keer trager dan de atomic-versie.

De contentions leiden niet tot een tragere werking dan de reduction optie. Het parallelisme van de GPU compenseert de synchronisatiekost, wat zorgt voor de beste uitvoeringstijd.

Bij de reductie zorgt de bijkomende synchronisatie en kernel overhead voor een hoger uitvoeringstijd dan bij de atomic-aanpak.

Bij kleine arrays wint de CPU vaak door lage latency. GPU's hebben hierbij last van launch- en driver-overhead. Bij grote arrays is de GPU duidelijk de beste, waarbij de atomic-versie in dit labo de snelste blijkt te zijn.

Samenvattend kunnen we besluiten dat GPU-parallelisatie een duidelijke prestatieverbetering oplevert ten opzichte van de sequentiële CPU-implementatie.